DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

¿Por qué estudiar Arquitectura de Computadores?

La era de la computación heterogénea

Sistemas Digitales

2do Cuatrimestre 2026

FCEN - UBA

Motivación

Motivación

¿Por qué estudiar **Arquitectura de Computadores**?

Motivación

¿Por qué estudiar **Arquitectura de Computadores**?

¿Por qué estudiar **Sistemas Digitales**?

Motivación: El impacto de la Jerarquía de Memoria

Acceso por filas (copyij)

```
void copyij(int src[2048][2048],
            int dst[2048][2048]) {
  int i, j;
  for (i = 0; i < 2048; i++)
    for (j = 0; j < 2048; j++)
      dst[i][j] = src[i][j];
}
```

Acceso por columnas (copyji)

```
void copyji(int src[2048][2048],
            int dst[2048][2048]) {
  int i, j;
  for (j = 0; j < 2048; j++)
    for (i = 0; i < 2048; i++)
      dst[i][j] = src[i][j];
}
```

Motivación: El impacto de la Jerarquía de Memoria

Acceso por filas (copyij)

```
void copyij(int src[2048][2048],
            int dst[2048][2048]) {
  int i, j;
  for (i = 0; i < 2048; i++)
    for (j = 0; j < 2048; j++)
      dst[i][j] = src[i][j];
}
```

4.3 ms

Acceso por columnas (copyji)

```
void copyji(int src[2048][2048],
            int dst[2048][2048]) {
  int i, j;
  for (j = 0; j < 2048; j++)
    for (i = 0; i < 2048; i++)
      dst[i][j] = src[i][j];
}
```

81.8 ms

Motivación: El impacto de la Jerarquía de Memoria

Acceso por filas (copyij)

```
void copyij(int src[2048][2048],
            int dst[2048][2048]) {
  int i, j;
  for (i = 0; i < 2048; i++)
    for (j = 0; j < 2048; j++)
      dst[i][j] = src[i][j];
}
```

4.3 ms

Acceso por columnas (copyji)

```
void copyji(int src[2048][2048],
            int dst[2048][2048]) {
  int i, j;
  for (j = 0; j < 2048; j++)
    for (i = 0; i < 2048; i++)
      dst[i][j] = src[i][j];
}
```

81.8 ms

2.0 GHz Intel Core i7 Haswell

Motivación: El impacto de la Jerarquía de Memoria

Acceso por filas (copyij)

```
void copyij(int src[2048][2048],
            int dst[2048][2048]) {
  int i, j;
  for (i = 0; i < 2048; i++)
    for (j = 0; j < 2048; j++)
      dst[i][j] = src[i][j];
}
```

4.3 ms

Acceso por columnas (copyji)

```
void copyji(int src[2048][2048],
            int dst[2048][2048]) {
  int i, j;
  for (j = 0; j < 2048; j++)
    for (i = 0; i < 2048; i++)
      dst[i][j] = src[i][j];
}
```

81.8 ms

2.0 GHz Intel Core i7 Haswell

- El desempeño depende de los patrones de acceso a la memoria

Motivación: Sistemas Digitales e...¿IA?

El caso de DeepSeek

- Redujeron 10x el tiempo/costo de entrenamiento respecto a modelos de OpenAI

Motivación: Sistemas Digitales e...¿IA?

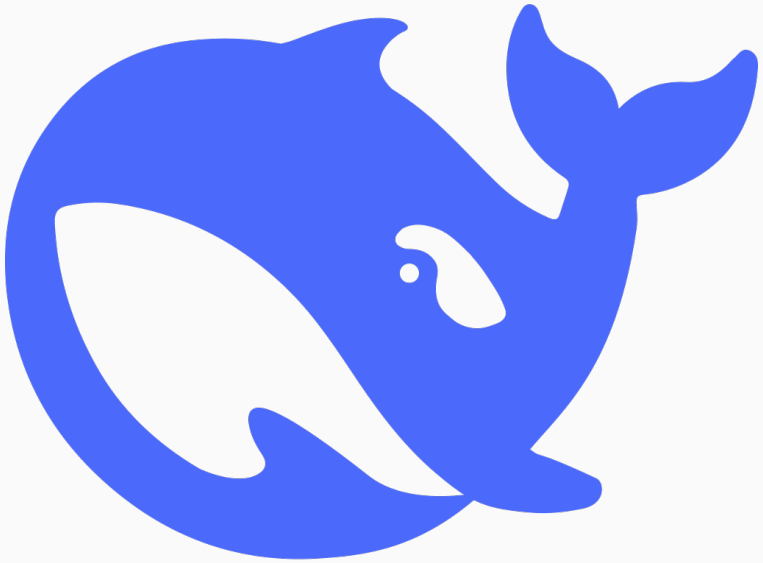
El caso de DeepSeek

- Redujeron 10x el tiempo/costo de entrenamiento respecto a modelos de OpenAI
- Optimizaciones usando el assembly de NVIDIA (PTX)

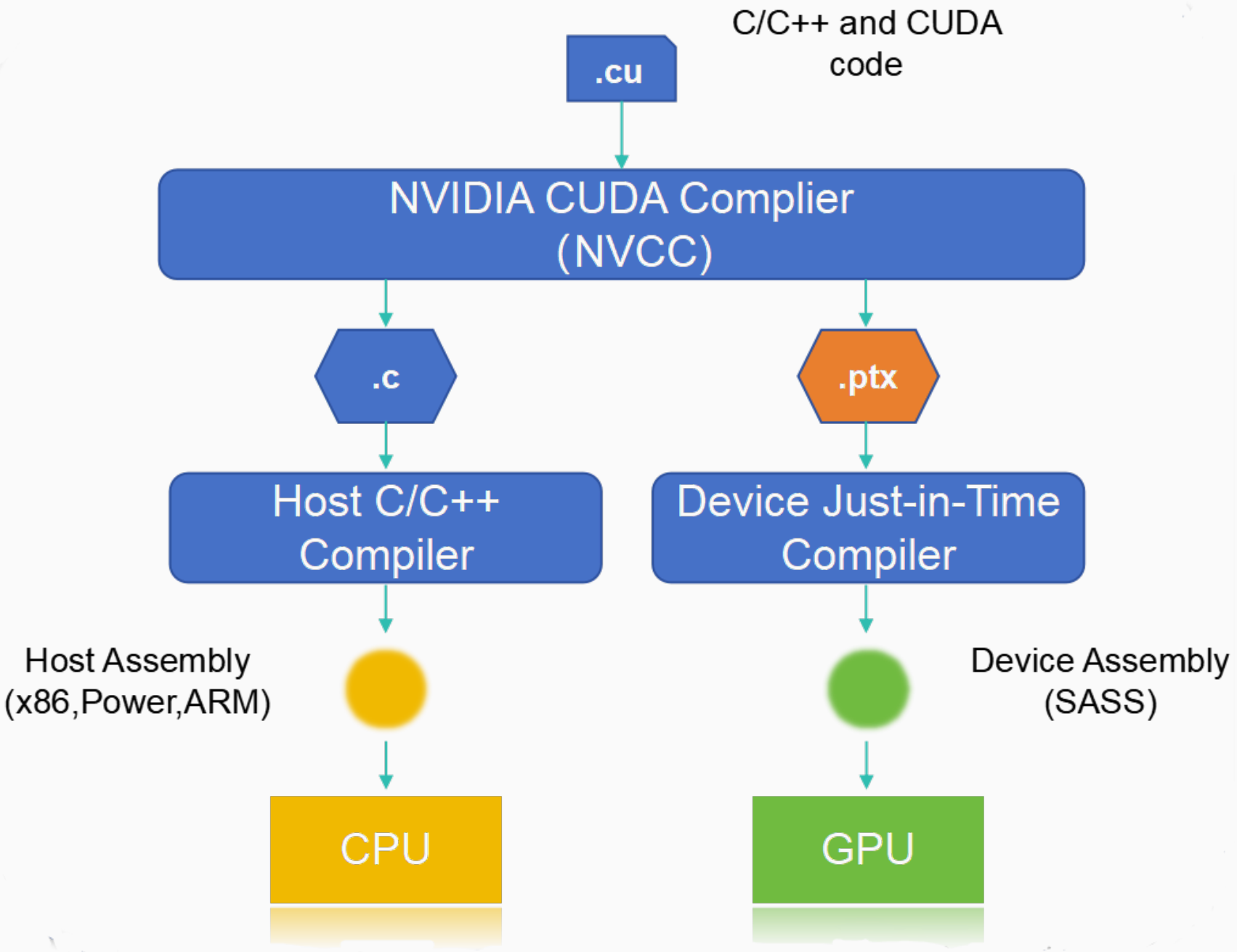
Motivación: Sistemas Digitales e...¿IA?

El caso de DeepSeek

- Redujeron 10x el tiempo/costo de entrenamiento respecto a modelos de OpenAI
- Optimizaciones usando el assembly de NVIDIA (PTX)
- Requiere un conocimiento profundo de la arquitectura de la GPU



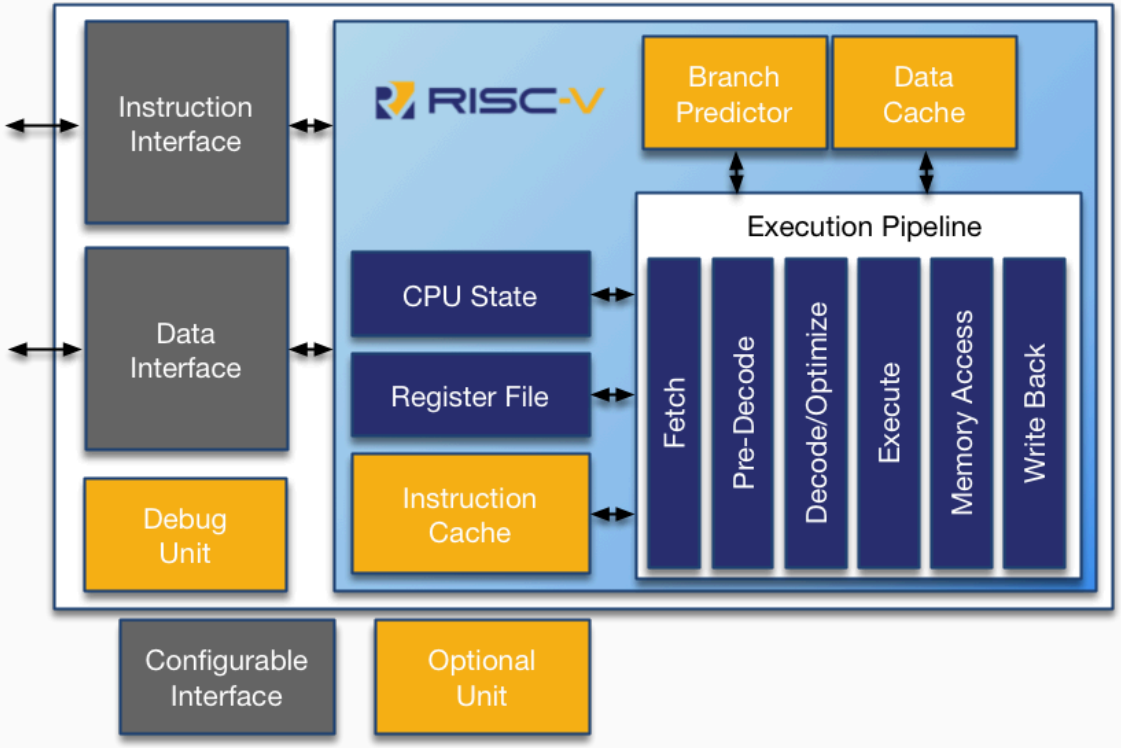
Motivación: Sistemas Digitales e...¿IA?



Motivación: Procesadores de “código abierto”

RISC-V

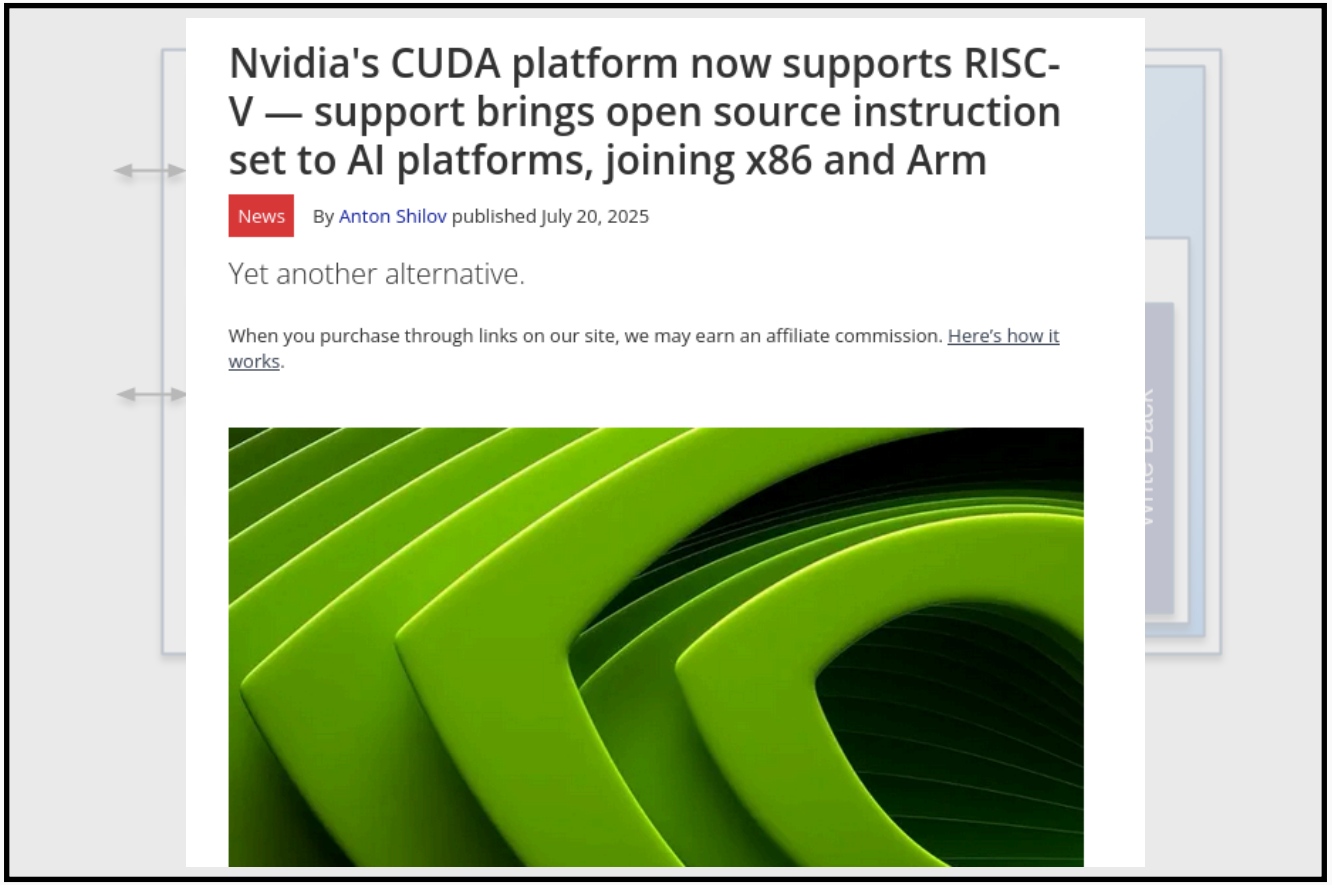
- Arquitectura abierta, libre, modular y extensible.



Motivación: Procesadores de “código abierto”

RISC-V

- Arquitectura abierta, libre, modular y extensible.



Nvidia's CUDA platform now supports RISC-V — support brings open source instruction set to AI platforms, joining x86 and Arm

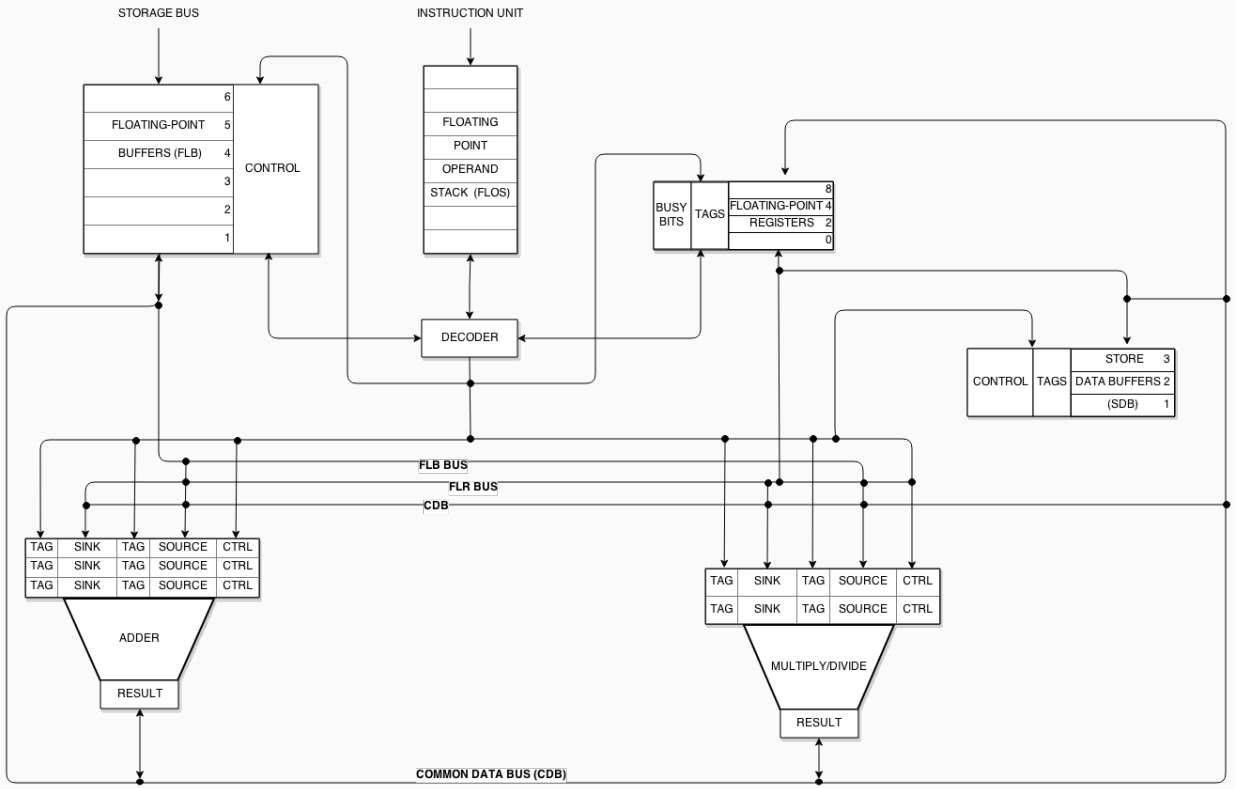
News By Anton Shilov published July 20, 2025

Yet another alternative.

When you purchase through links on our site, we may earn an affiliate commission. Here's how it works.

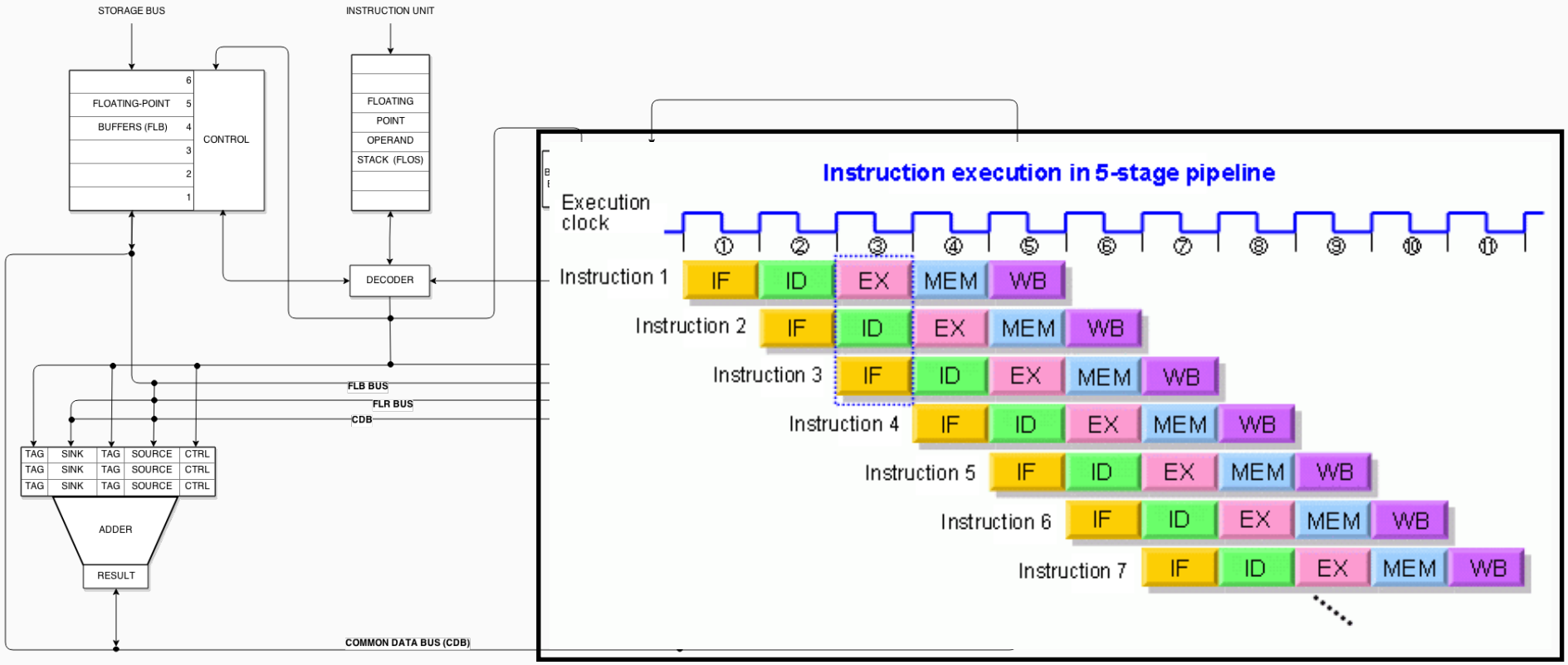
Motivación: El hardware “optimiza” el software

- Arquitecturas superscalares, Tomasulo, Pipeline, Predicción de saltos.



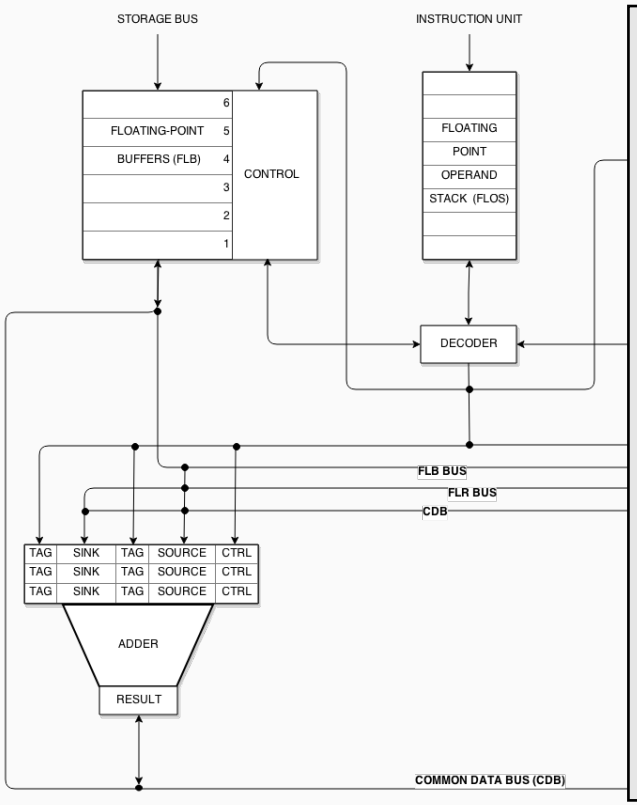
Motivación: El hardware “optimiza” el software

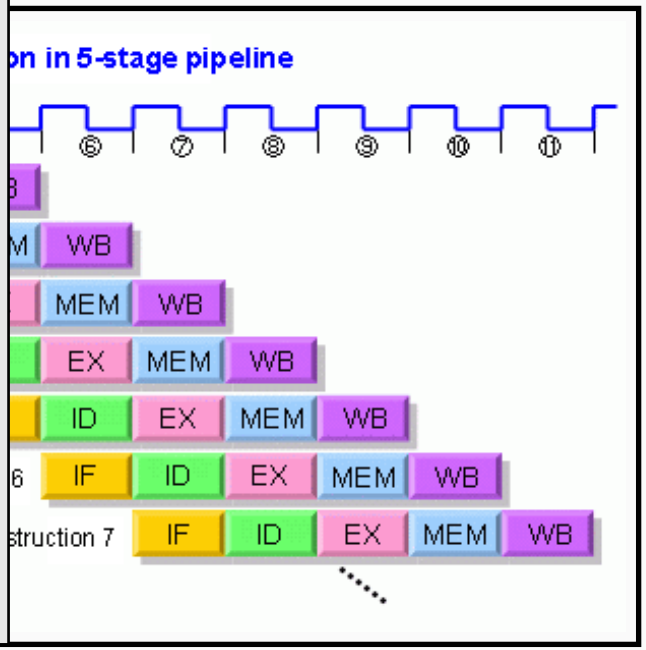
- Arquitecturas superscalares, Tomasulo, Pipeline, Predicción de saltos.



Motivación: El hardware “optimiza” el software

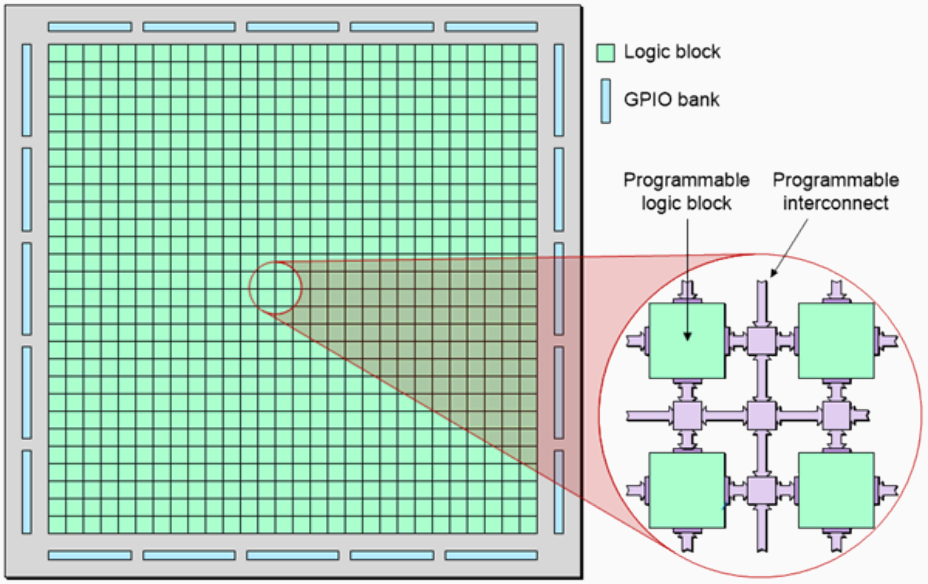
- Arquitecturas superscalares, Tomasulo, Pipeline, Predicción de saltos.





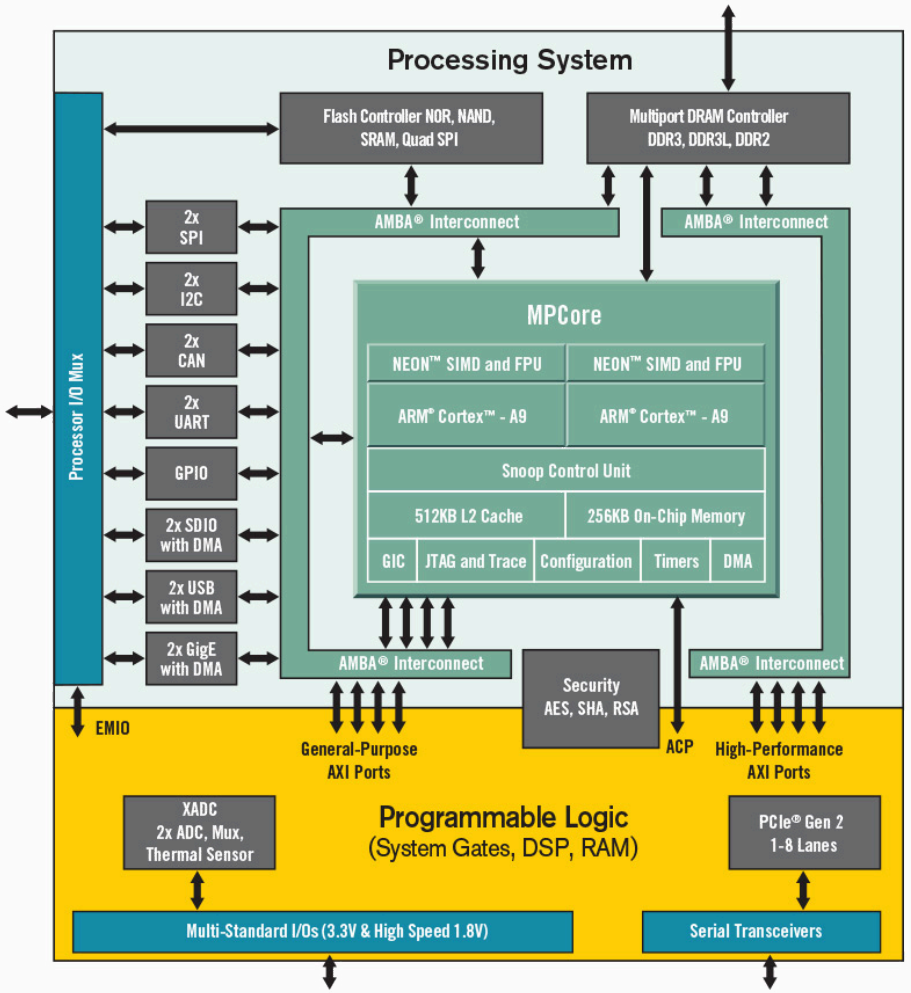
Motivación: Silicio reconfigurable

FPGA Bird-eye



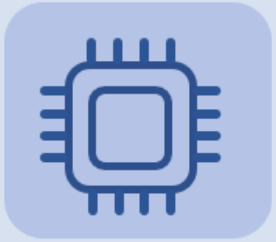
Bird's-eye view of FPGA

Zynq SoC



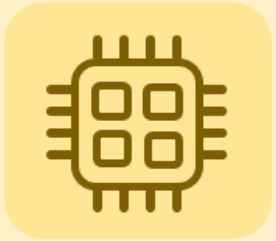
- *Mar de compuertas* reconfigurable + CPUs potentes.

Motivación: Computación heterogénea e Impacto Local



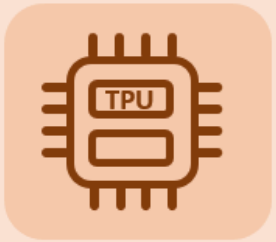
CPU

- Small models
- Small datasets
- Useful for design space exploration



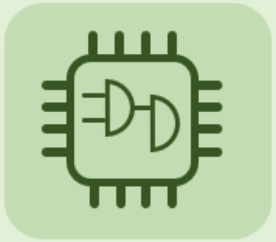
GPU

- Medium-to-large models, datasets
- Image, video processing
- Application on CUDA or OpenCL



TPU

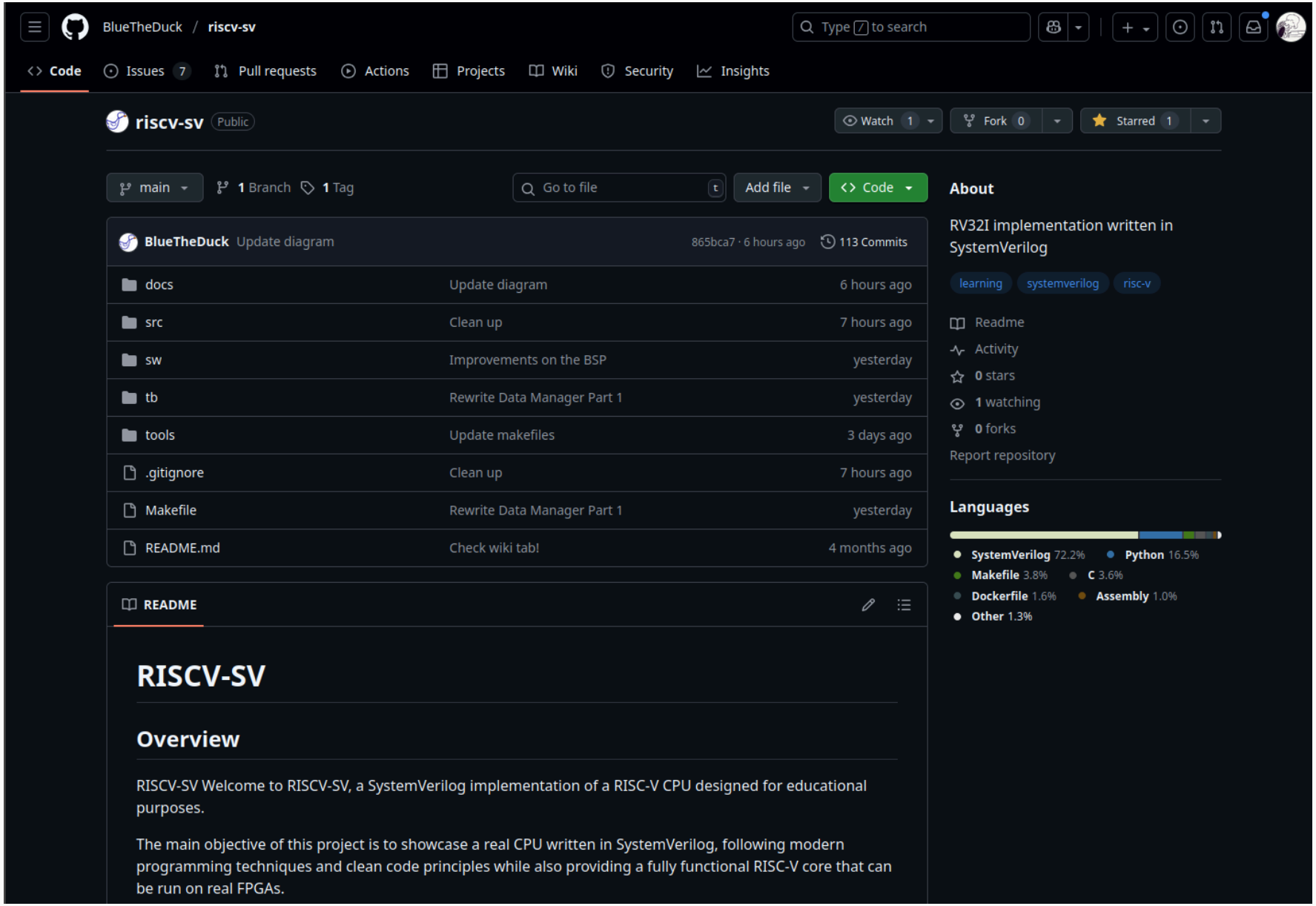
- Matrix computations
- Dense vector processing
- No custom TensorFlow operations



FPGA

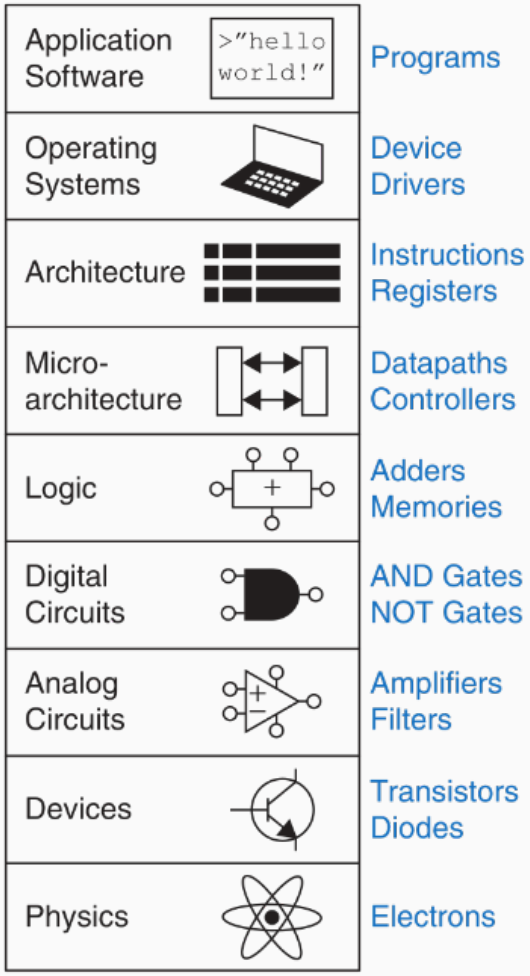
- Large datasets, models
- Compute intensive applications
- High performance, high perf./cost ratio

Motivación: Al alcance de nuestras manos

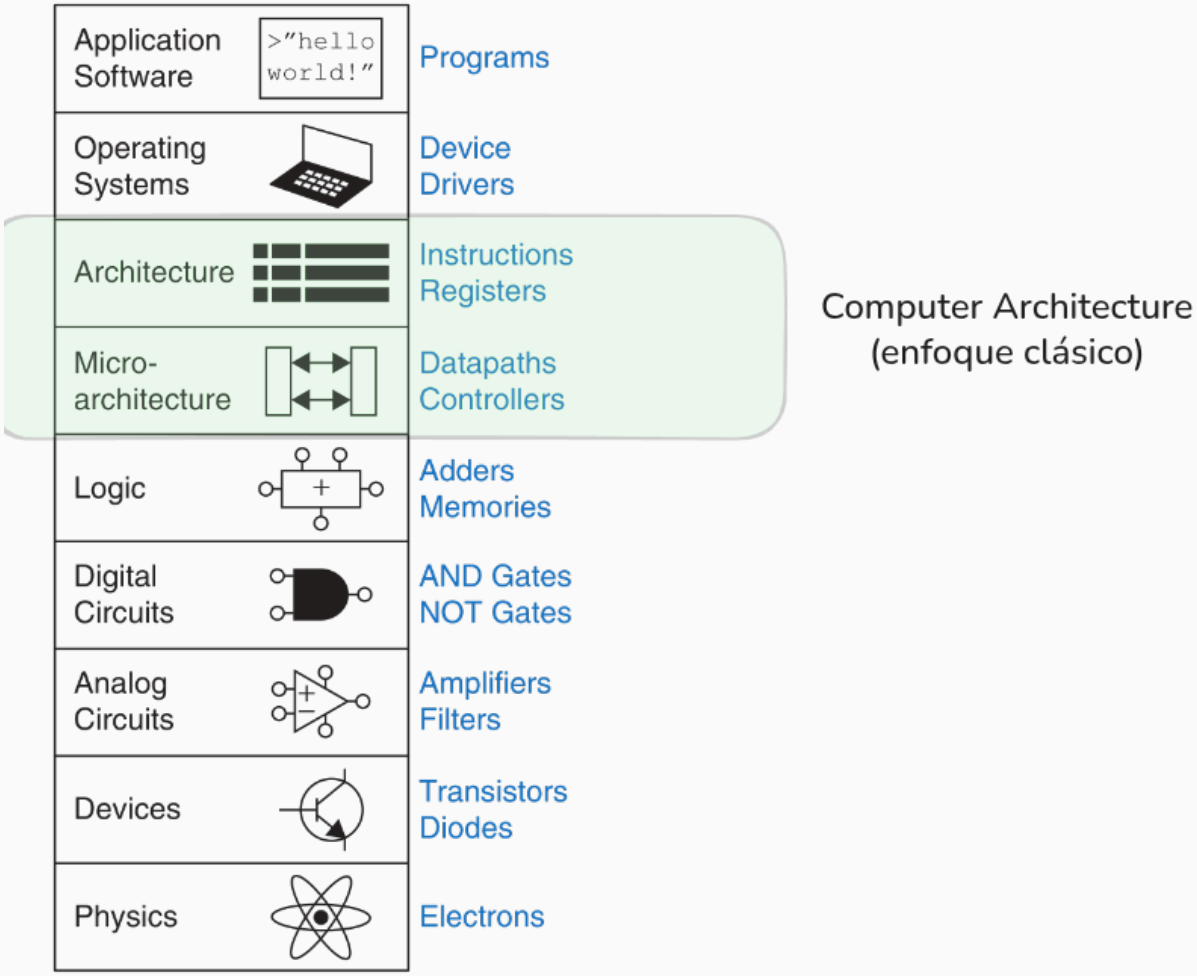


RISC-V implementado por Pedro Perez como final de AyOC: https://github.com/BlueTheDuck/riscv-sv

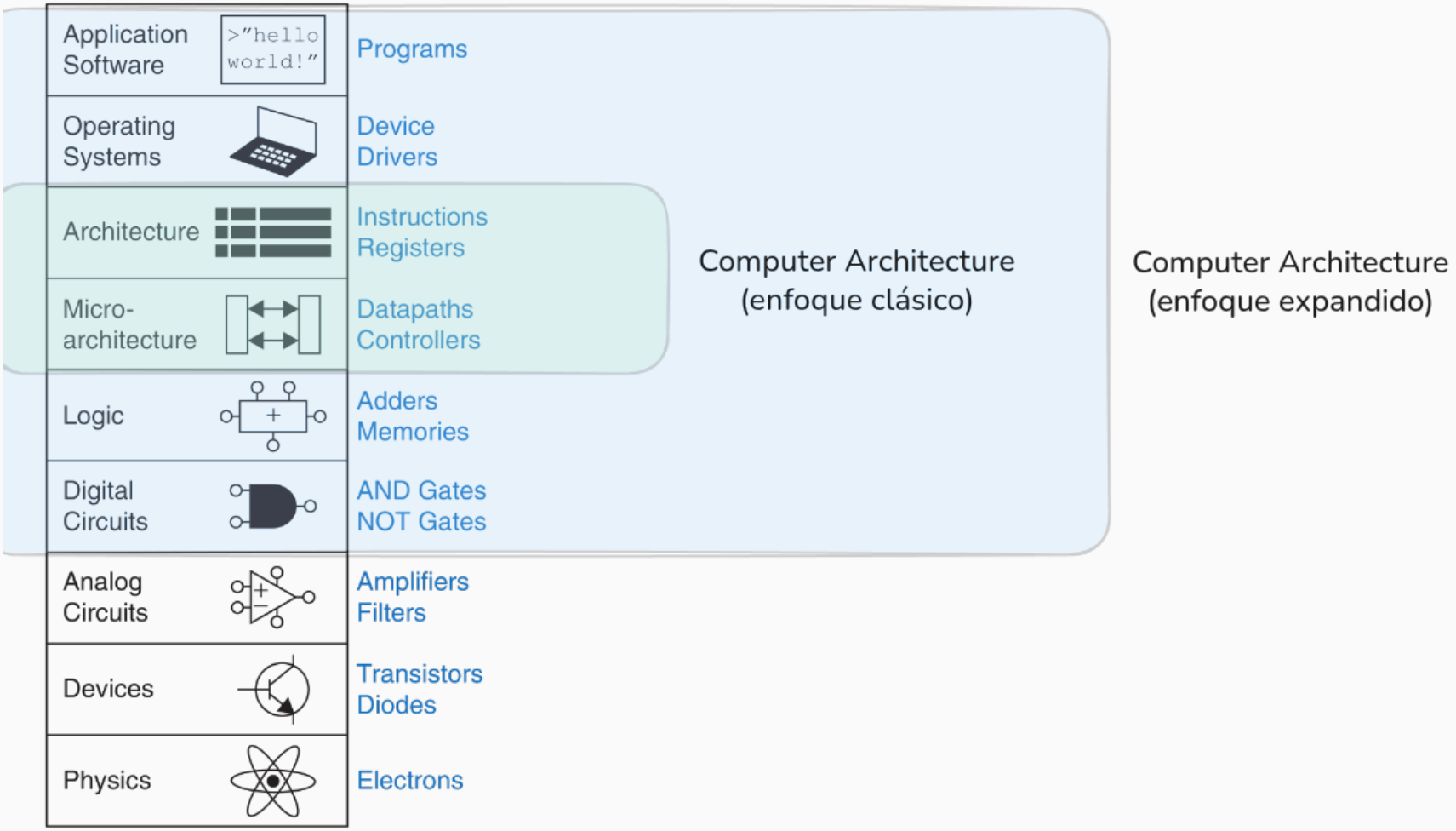
Jerarquía de sistemas de cómputo



Jerarquía de sistemas de cómputo



Jerarquía de sistemas de cómputo



Arquitectura del computador hoy

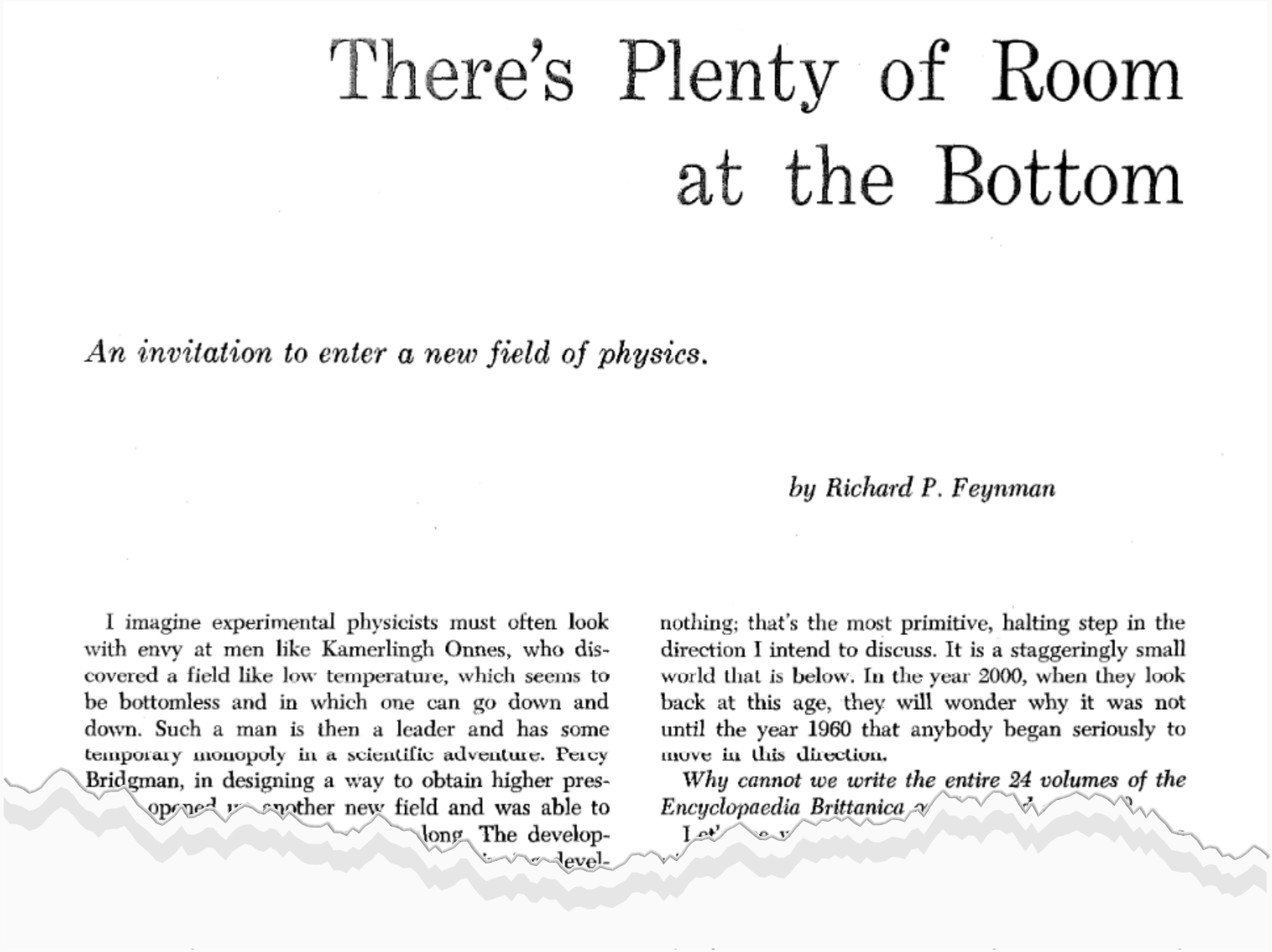
Estudiar arquitectura hoy es más importante que antes

La industria está cambiando

- *Data intensive* (IA), restricciones de energía.
- Cuellos de botella en memoria, seguridad y privacidad.

No hay una única solución: GPUs, TPUs, FPGAs, RISC-V...

Espacios para mejoras



There's Plenty of Room at the Bottom

An invitation to enter a new field of physics.

by Richard P. Feynman

I imagine experimental physicists must often look with envy at men like Kamerlingh Onnes, who discovered a field like low temperature, which seems to be bottomless and in which one can go down and down. Such a man is then a leader and has some temporary monopoly in a scientific adventure. Percy Bridgman, in designing a way to obtain higher pres- opened [illegible] another new field and was able to [illegible] long. The develop- [illegible] devel- nothing; that's the most primitive, halting step in the direction I intend to discuss. It is a staggeringly small world that is below. In the year 2000, when they look back at this age, they will wonder why it was not until the year 1960 that anybody began seriously to move in this direction.

Why cannot we write the entire 24 volumes of the Encyclopaedia Brittanica [illegible]

Richard Feynman: "There's Plenty of Room at the Bottom"

Fuente: https://calteches.library.caltech.edu/1976/1/1960Bottom.pdf

Espacios para mejoras

REVIEW SUMMARY

COMPUTER SCIENCE

There's plenty of room at the Top: What will drive computer performance after Moore's law?

Charles E. Leiserson, Neil C. Thompson*, Joel S. Emer, Bradley C. Kuszmaul, Butler W. Lampson, Daniel Sanchez, Tao B. Schardl

BACKGROUND: Improvements in computing power can claim a large share of the credit for many of the things that we take for granted in our modern lives: cellphones that are more powerful than room-sized computers from 25 years ago, internet access for nearly half the world, and drug discoveries enabled by powerful supercomputers. Society has come to rely on computers whose performance increases exponentially over time.

Much of the improvement in computer performance comes from decades of miniaturization of computer components, a trend that was foreseen by the Nobel Prize–winning physicist Richard Feynman in his 1959 address, "There's Plenty of Room at the Bottom," to the American Physical Society. In 1975, Intel founder Gordon Moore predicted the regu- [illegible] of this miniaturization trend, now called [illegible] recently, doubled the [illegible]

in computing power stalls, practically all industries will face challenges to their productivity. Nevertheless, opportunities for growth in computing performance will still be available, especially at the "Top" of the computing-technology stack: software, algorithms, and hardware architecture.

ADVANCES: Software can be made more efficient by performance engineering: restructuring software to make it run faster. Performance engineering can remove inefficiencies in programs, known as software bloat, arising from traditional software-development strategies that aim to minimize an application's development time rather than the time it takes to run. Performance engineering can also tailor software to the hardware on which it runs, for example, to take advantage of parallel processors and vector units.

Algorithms offer more-efficient [illegible] Indeed, sin[illegible]

and sporadically and must ultimately face diminishing returns. As such, we see the biggest benefits coming from algorithms for new problem domains (e.g., machine learning) and from developing new theoretical machine models that better reflect emerging hardware.

ON OUR WEBSITE

Read the full article at https://dx.doi.org/10.1126/science.aam9744

Hardware architectures can be streamlined—for instance, through processor simplification, where a complex processing core is replaced with a simpler core that requires fewer transistors. The freed-up transistor budget can then be redeployed in other ways—for example, by increasing the number of processor cores running in parallel, which can lead to large efficiency gains for problems that can exploit parallelism. Another form of streamlining is domain specialization, where hardware is customized for a particular application domain. This type of specialization jettisons processor functionality that is not needed for the domain. It can also allow more customization to the specific characteristics of the domain, for instance, by decreasing floating-point precision for machine-learning applications.

In the post-Moore era, performance improvements from software, algorithms, and hardware architecture will increasingly require concurrent changes across other levels of the stack. These changes will be easier to im- [illegible] ent and [illegible] within

Fuente: https://www.science.org/doi/epdf/10.1126/science.aam9744

Hardware vs Software

Cada vez más difícil optimizar software sin conocer hardware

Hardware vs Software

Cada vez más difícil optimizar software sin conocer hardware

- Mejor software conociendo el hardware.
- Mejor hardware conociendo el software.
- **Mejores sistemas de cómputo** conociendo ambos.

Objetivo de la materia

- **Sistemas de cómputo digitales**

Objetivo de la materia

- **Sistemas de cómputo digitales**
- **Microarquitectura**: Construcción a partir de bloques digitales.

Objetivo de la materia

- **Sistemas de cómputo digitales**
- **Microarquitectura**: Construcción a partir de bloques digitales.
- **Arquitectura**: Instrucciones, registros y memoria.